HARNESS LAB · MÓDULO 1 · AULA 1.4

As 4 camadas *do harness*

Cada falha que você viu tem uma camada que resolve. Regra informa, Trava impede, Prova confere, Loop repete. Nenhuma das quatro depende de o modelo ser mais esperto.

`Regra` `Trava` `Prova` `Loop` `CLAUDE.md` `AGENTS.md` `skills` `hooks` `exit 2` `permissions.deny` `claude -p` `/diagnostico-camadas`

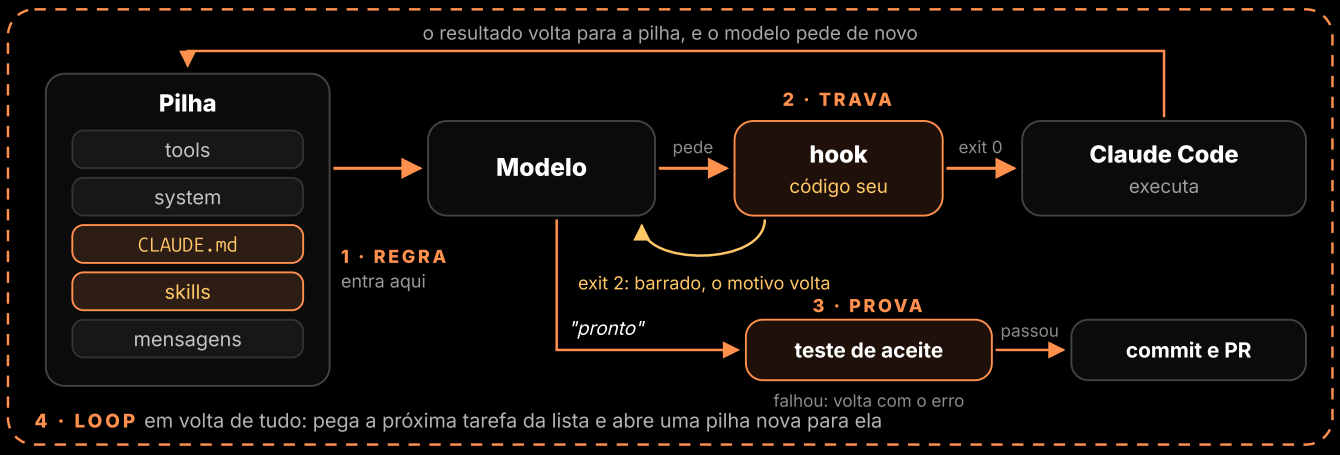
VOCÊ SAI COM

O `diagnostico-camadas.md` no seu repositório: as 5 correções da aula 1.3, cada uma com a camada que faltou, a contagem e a camada por onde começar.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula volta à pilha da 1.2. Tools, system prompt, CLAUDE.md, skills e mensagens vão para o modelo a cada volta. O modelo pede uma ferramenta, o Claude Code executa, o resultado volta para a pilha, e o loop segue. O harness são quatro camadas encaixadas nesse loop.



Regra já vem no ferramental: CLAUDE.md, AGENTS.md, skills, subagentes, todo o contexto que você escreve em markdown. **Trava** são os hooks: código determinístico que bloqueia a execução destrutiva, como um `migrate:fresh` ou um comando que apaga o arquivo errado. O mesmo mecanismo faz coisas a mais, como avisar no WhatsApp que a tarefa terminou. **Prova** são os gates: passou nos testes, a qualidade está boa, o estilo é o do projeto? Só aí segue para o commit e o PR. Todo agente precisa de um avaliador. **Loop** quebra a tarefa, por maior que seja, em pedaços do tamanho ótimo para a janela do modelo, e trabalha incremental.

O curso começa pela Trava, no Módulo 2: é a parte mais determinística, mais código e pouco texto, e é a primeira vitória. Banco apagado, medo de rodar algo em produção, variável de ambiente lida sem querer: tudo isso para logo ali. A prática fecha a aula: pegue as correções da 1.3 e o `git log`, e classifique cada falha. Faltou regra, trava, prova ou loop?

Regra informa

O que ele precisa saber, escrito em arquivo: CLAUDE.md, AGENTS.md, skills, subagentes.

Trava impede

Roda fora do modelo. Ele não decide se a trava vale: o código de saída decide.

Prova confere

Quem diz "pronto" é o teste, não o modelo. Suíte verde não é integração real.

Loop repete

Quebra a tarefa, por maior que seja, num tamanho ótimo para o modelo e trabalha incremental.

> *"Claude treats them as context, not enforced configuration. To block an action regardless of what Claude decides, use a PreToolUse hook instead."*
>
> Documentação oficial do Claude Code · How Claude remembers your project (sobre CLAUDE.md e auto memory)

2 Indo *além da aula*

ALÉM DA AULA · AGENTS.MD É "A MESMA COISA", COM UMA CONDIÇÃO

O Claude Code 2.1.277 ou mais novo lê o `AGENTS.md` sozinho, mas só quando não existe `CLAUDE.md` nem `CLAUDE.local.md` na pasta ou acima dela. Se o repositório tem os dois, ele lê só o CLAUDE.md, e a regra que está no AGENTS.md some da pilha sem aviso. Para usar um arquivo só em todas as ferramentas, a doc indica pôr `@AGENTS.md` na primeira linha do CLAUDE.md e o que for só do Claude embaixo.

O REPOSITÓRIO TEM	O CLAUDE LÊ
Só AGENTS.md	O AGENTS.md
AGENTS.md e CLAUDE.md	Só o CLAUDE.md
CLAUDE.md com `@AGENTS.md`	Os dois: o AGENTS.md entra pelo import

Fonte: How Claude remembers your project, seções "AGENTS.md" e "Share one file with other coding tools".

ALÉM DA AULA · TRAVA SEM ESCREVER SCRIPT

Para o caso do vídeo, "ler uma variável de ambiente que não deveria", a trava mais simples nem é hook: é uma regra `deny` no `.claude/settings.json`, como `Read(./.env)`. O próprio Claude Code aplica a regra, faça o modelo o que fizer. Ela cobre as ferramentas de arquivo e os comandos que ele reconhece no terminal (`cat`, `head`, `tail`, `sed`). Não cobre um `grep -r` que lê sem citar o arquivo, nem um script Python ou Node que abre o arquivo por conta própria. Para bloquear no nível do sistema operacional, a doc indica o sandbox. Hook é para quando a decisão tem lógica: casar um comando, explicar o motivo, liberar o resto.

Fonte: Configure permissions, "Read and Edit" e o aviso em "What Read and Edit rules cover".

ALÉM DA AULA · CAMADA É FUNÇÃO, NÃO ARQUIVO

O mesmo hook serve a camadas diferentes. Um `PreToolUse` que barra `migrate:fresh` é Trava. Um `Stop` que roda o teste de aceite e devolve `exit 2` é Prova: na doc, o exit 2 no Stop "prevents Claude from stopping" e ele continua trabalhando. Um `SessionStart` que injeta contexto é Regra. Na hora de classificar uma correção, não pergunte "isso se resolve com qual arquivo?". Pergunte o que faltou: informar, obrigar, conferir ou continuar. E a Prova só vale se o teste toca o que importa: a suíte verde que não chama a integração real diz "pronto" do mesmo jeito que o modelo.

Fonte: Hooks reference, tabela "Exit code 2 behavior per event" e "Decision control".

ALÉM DA AULA · O LOOP COM CLAUDE -P: TRÊS DETALHES QUE TRAVAM A PRIMEIRA VEZ

Cada `claude -p` é uma sessão nova: é isso que dá a pilha limpa por tarefa. Só que, a partir do 2.1.302, o `-p` começa no modo Manual quando nada define o modo, e no modo Manual toda chamada que pediria aprovação é negada. Sem `--permission-mode acceptEdits` (ou `--allowedTools`), o loop roda e não edita nada. O comando de teste também precisa estar liberado. Segundo: não use `--bare` para "ficar mais rápido". Ele pula CLAUDE.md, hooks, skills e plugins: o loop roda sem Regra e sem Trava. Terceiro: no `-p` não aparece a tela de confiança na pasta, e os hooks do `.claude/settings.json` rodam direto. Revise o `.claude/` de um repositório que não é seu antes de pôr num loop.

`modelos/mapa-das-camadas/loop.sh · trecho` — esboço testado · o seu nasce no Módulo 5

```
while tarefa=$(proxima) && [ -n "$tarefa" ]; do
  feito=0
  for tentativa in $(seq 1 "$TENTATIVAS"); do
    # sessão nova; sem --permission-mode o -p nega as edições; sem --bare
    claude -p "Implemente só esta tarefa: $tarefa. Siga o CLAUDE.md." \
      --permission-mode acceptEdits --allowedTools "Bash($TESTE *)"
    if $TESTE; then   # a Prova decide, não o modelo
      marcar_feita "$tarefa"; git add -A && git commit -qm "$tarefa"; feito=1; break
    fi
  done
  [ "$feito" = 1 ] || { avisar "travou em: $tarefa"; exit 1; }
done
```

Por que uma tarefa pequena com Prova em cada uma, numa conta (não estudo): 20 passos com 95% de acerto cada, sem ninguém conferir no meio, terminam certos em $0{,}95^{20} \approx 36\%$ das vezes. Conferindo a cada tarefa, o erro é pego onde nasceu.

Fonte: Run Claude Code programmatically, "Auto-approve tools" e "Start faster with bare mode"; Hooks reference, "Workspace trust".

ALÉM DA AULA · O AVISO NO WHATSAPP: QUAL EVENTO ESCUTAR

O `Stop` dispara no fim de toda resposta, não só quando a tarefa acabou: vira spam no celular. Para "ele parou e está esperando você", o evento é `Notification`, com o matcher `idle_prompt` (resposta pronta há uns 60 segundos e você não digitou) ou `permission_prompt` (pedido de permissão esperando há uns 6 segundos). Notification não bloqueia nada: o código de saída é ignorado. Para o WhatsApp, o hook manda um POST para um webhook seu (n8n, Make, um bot). O `avisa.sh` do mapa faz isso com a variável `AVISO_WEBHOOK`, e sem ela usa a notificação do sistema. No loop com `claude -p` não existe "esperando você": quem avisa é o próprio script, no fim.

Fonte: Hooks reference, "Notification" e a tabela de matchers.

3 Faça *no seu repositório*

O que entra no seu projeto: o `diagnostico-camadas.md`, na raiz, com as perguntas e a tabela vazia. Fora do projeto: o plugin `diagnostico-camadas`, com a skill que faz a classificação com você. Nada seu é apagado. Leva uns 10 minutos.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Crie o diagnóstico no seu projeto.

O passo 1 terminou em `~`. Antes deste comando, entre na pasta do seu projeto com `cd`: o instalador se recusa a rodar na pasta pessoal.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 1.4
```

A saída mostra `criado diagnostico-camadas.md`. Não quer versionar? Ponha o nome no `.gitignore`.

3 Instale o plugin da aula.

Terminal · qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install diagnostico-camadas@harness-lab
claude plugin list
```

O `update` importa para quem adicionou o marketplace numa aula anterior: o `add` responde "already on disk" e não traz o plugin novo. O `claude plugin list` tem que mostrar `diagnostico-camadas@harness-lab`. Prefere fazer de dentro do Claude Code? O mesmo, em comandos de sessão:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install diagnostico-camadas@harness-lab
/reload-plugins
```

4 **Junte as evidências.**

A lista principal são as 5 correções que você anotou na aula 1.3 (o `correcoes.md`, se usou o plugin daquela aula). O `git log` ajuda a lembrar o que você corrigiu esta semana:

Terminal · na raiz do seu projeto — shell

```
git log --oneline -15
```

5 **Rode a classificação no Claude Code.**

Abra o Claude Code na raiz do projeto. A skill lê o `correcoes.md` (ou o arquivo que você passar), roda as evidências do git, faz as perguntas abaixo para cada correção e pergunta para você quando não dá para saber. No fim, preenche o `diagnostico-camadas.md`.

Dentro do Claude Code — sessão

```
/diagnostico-camadas
# ou apontando o arquivo das correções
/diagnostico-camadas correcoes.md
```

Sem o plugin, o pedido em texto faz o mesmo trabalho, com menos estrutura:

Dentro do Claude Code — sessão

```
> Classifique minhas 5 correções da 1.3 em Regra, Trava, Prova ou Loop
```

6 **Confira cada linha com as perguntas, nesta ordem.**

Pare no primeiro "sim". Quem estava lá é você: se a skill marcou errado, corrija.

#	PERGUNTA	SE SIM, FALTOU
1	Ele não tinha a informação que precisava?	Regra: informar
2	Ele sabia (recita se você perguntar) e fez mesmo assim?	Trava: obrigar
3	Ele disse "pronto" e não estava?	Prova: conferir
4	Ele parou e esperou você, ou a sessão longa estragou o resto?	Loop: seguir sozinho
5	Com as quatro de pé, ele ainda erraria?	Inteligência: modelo

A quinta pergunta existe e costuma ficar quase vazia. Antes de marcar "Inteligência", diga por que nenhuma das quatro resolveria. Dois exemplos resolvidos (mais um por camada em `references/exemplos.md` do plugin):

O QUE ACONTECEU	CAMADA	POR QUÊ
O CLAUDE.md dizia "nunca rode `migrate:fresh`". No meio da feature, para o teste passar, ele rodou e apagou o banco local. (Node: `npx prisma migrate reset`; Rails: `rails db:reset`.)	Trava	Ele sabia e fez mesmo assim. Mais regra não resolve; o hook que barra resolve (aula 2.3)
Usou um método que não existe mais na versão do SDK do projeto. Parecia "o modelo é burro".	Regra	Ele não tinha a informação: conhece a versão que viu no treino. A doc da sua versão entra como skill (Módulo 3)

7 Veja o resultado.

Terminal · na raiz do seu projeto — shell

```
sed -n '/^## Contagem/,$p' diagnostico-camadas.md
```

A camada com mais correções é a que mais faz falta no seu projeto hoje. Se for Trava, ótimo: é o Módulo 2, logo depois deste.

O que a skill faz por baixo

`plugins/diagnostico-camadas/skills/diagnostico-camadas/SKILL.md · trecho` — instalado pelo plugin · não copie do PDF

```
---
name: diagnostico-camadas
disable-model-invocation: true          # só roda quando você chama
allowed-tools: Bash(${CLAUDE_SKILL_DIR}/scripts/evidencias.sh *)
---
1. Junte as correções: $ARGUMENTS, correcoes.md, ou pergunte as 5 ao usuário.
   Rode ${CLAUDE_SKILL_DIR}/scripts/evidencias.sh 100 (commits de correção,
   reverts, arquivos que mais voltaram). Git completa a lista, nunca inventa.
2. Classifique: as 5 perguntas na ordem, pare no primeiro "sim", pela causa.
   Na dúvida entre Regra e Trava, procure a regra no CLAUDE.md e em .claude/rules/.
3. Grave diagnostico-camadas.md: tabela, contagem, por onde começar.
   Não crie hook, skill nem regra agora: cada camada é montada no módulo dela.
```

O `evidencias.sh` só lê o git (testado: o status e o HEAD não mudam). O `allowed-tools` libera exatamente esse script, então ele roda sem pedir permissão.

4 Arquivos que *reforçam a aula*

O `modelos/mapa-das-camadas/` é um repositório Laravel de mentira com as quatro camadas no lugar onde moram, os mesmos arquivos dos slides. Não é instalado: é para abrir e ver onde cada coisa fica.

`CLAUDE.md` e a skill `api-padroes`

Regra: o `CLAUDE.md` curto (stack, comandos, o que nunca fazer) aponta para `.claude/skills/api-padroes/SKILL.md`. Da skill, só o nome e a descrição entram em toda chamada.

No repositório: aula-1.4-as-4-camadas-do-harness/modelos/mapa-das-camadas/

`.claude/settings.json` e `bloqueia.sh`

Trava: o `deny` do `.env` sem script e o `PreToolUse` que barra `migrate:fresh`, `rm -rf` e `push --force` com `exit 2`. Esboço: a versão completa é a da aula 2.3.

No repositório: aula-1.4-as-4-camadas-do-harness/modelos/mapa-das-camadas/.claude/

`.claude/hooks/avisa.sh`

A "feature a mais" do vídeo: `Notification` que avisa no desktop e, com `AVISO_WEBHOOK`, no celular ou no WhatsApp pelo seu webhook.

No repositório: aula-1.4-as-4-camadas-do-harness/modelos/mapa-das-camadas/.claude/hooks/

`specs/tarefa-01.md` e `ProdutosTest.php`

Prova: três critérios de aceite escritos antes do código, e cada um virou um teste Pest em `tests/Feature/`. Em outra stack, o mesmo: critério, teste, código de saída.

No repositório: aula-1.4-as-4-camadas-do-harness/modelos/mapa-das-camadas/

`loop.sh` e `tarefas.md`

Loop: pega a primeira tarefa sem `[x]`, sessão nova, a Prova decide, commit, até 3 tentativas, avisa que travou ou terminou. Testado com um `claude` falso.

No repositório: aula-1.4-as-4-camadas-do-harness/modelos/mapa-das-camadas/

`references/exemplos.md`

Um caso resolvido por camada, os dois que mais confundem e a lista "sintoma não é causa". Para treinar o olho antes de classificar as suas.

No repositório: plugins/diagnostico-camadas/skills/diagnostico-camadas/references/

Para ver a Trava responder sem abrir o Claude Code: o hook é só um programa, entra JSON no stdin e sai código. Ele precisa do `jq` (`apt install jq` ou `brew install jq`).

Terminal · qualquer pasta — shell

```
cd ~/harness-lab-materiais/modulo-1-*/aula-1.4-*/modelos/mapa-das-camadas
j='{"tool_input":{"command":"php artisan migrate:fresh"}}'
echo "$j" | .claude/hooks/bloqueia.sh || echo "exit $?"
```

O esperado: `Bloqueado pelo hook: php artisan migrate:fresh...` e `exit 2`. Troque por `php artisan migrate` e nada aparece: é o `exit 0`.

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Quase tudo caiu em "Inteligência"	É a explicação mais fácil e não pede mudança nenhuma no projeto	Faça as quatro perguntas antes da quinta. Inteligência é o que sobra
Caso de Trava resolvido com mais regra no CLAUDE.md	Parece natural reforçar em maiúscula. Mas ele já sabia	Se ele recita a regra e faz mesmo assim, a resposta é obrigar (hook ou `deny`)

SINTOMA	CAUSA	CORREÇÃO
"Errou o teste" marcado como Prova	Classificou pelo sintoma	Pergunte o que teria evitado. Se ele não sabia como o projeto testa, é Regra
`Plugin "diagnostico-camadas" not found in marketplace "harness-lab"`	Marketplace adicionado numa aula anterior: o `add` não atualiza a cópia local	`claude plugin marketplace update harness-lab` e instale de novo
O teste do `bloqueia.sh` mostra "instale o jq" e `exit 2`	Falta o `jq`. O exit 2 aqui não prova que a Trava funcionou	`apt install jq` ou `brew install jq` e rode de novo
`/diagnostico-camadas` não aparece	Plugin instalado com a sessão aberta	`/reload-plugins`, ou abra uma sessão nova. Confira com `claude plugin list`
Regra do AGENTS.md ignorada	O repositório também tem CLAUDE.md, e aí só ele é lido	`@AGENTS.md` na primeira linha do CLAUDE.md (seção 2)
Loop com `claude -p` roda e não muda nada	No modo Manual, o `-p` nega as edições que pediriam aprovação	`--permission-mode acceptEdits` e o teste em `--allowedTools`
O `bloqueia.sh` deixou passar `rm -r -f`	Testado: regex simples não pega toda variação. É o limite do esboço	Use a versão da aula 2.3, que trata as variações e falha fechado

6 Você sai *com*

- [ ] `diagnostico-camadas.md` na raiz do projeto, com as 5 correções e uma camada em cada.
- [ ] A contagem por camada e a camada por onde começar, com uma frase sua do que ela resolveria.
- [ ] O plugin `diagnostico-camadas` em `claude plugin list`.
- [ ] Para cada camada, onde ela mora no repositório e em que módulo você monta a sua.
- [ ] O `bloqueia.sh` do mapa devolvendo `exit 2` para `migrate:fresh` no seu terminal.

PROMETIDO NA AULA

- ✓ "Dá uma olhada nesse slide aqui e roda lá para o seu Claude Code para a gente fazer uma classificação": as perguntas estão na seção 3, passo 6, e a skill `/diagnostico-camadas` faz a classificação (passo 5).
- ✓ "Pega aí o seu log do git também": `git log --oneline -15` no passo 4 e o `evidencias.sh`, que a skill roda sozinha.
- ✓ "Vou avisar no WhatsApp lá que eu finalizei. É possível fazer isso através de hooks": `modelos/mapa-das-camadas/.claude/hooks/avisa.sh` com `AVISO_WEBHOOK` (seções 2 e 4).
- ✓ "A gente vai entender como que a gente proíbe comandos": o esboço `bloqueia.sh` está no mapa (seção 4); a versão completa é a aula 2.3.

- ✓ "A gente vai tudo expandir no momento certo": o módulo de cada camada está na contagem do `diagnostico-camadas.md` e em cada arquivo do mapa.
- ✓ "Seu diagnóstico em diagnostico-camadas.md" (slide): o `instalar.sh 1.4` cria o arquivo na raiz do projeto (seção 3, passo 2).
- ✓ "Vamos escolher um alvo também na próxima aula": caixa Próxima, no fim, Aula 1.5 · Escolha sua feature-alvo.

7 Glossário

Harness	Regra, Trava, Prova e Loop em volta do agente. O trilho que faz ele trabalhar do seu jeito sem você olhar cada passo.
Regra	O que ele precisa saber, escrito em arquivo: CLAUDE.md, AGENTS.md, skills, subagentes.
Trava	Bloqueio que roda fora do modelo: hook ou regra de permissão.
Prova (gate)	O teste que decide se terminou, a partir de um critério de aceite seu.
Loop	Pega a próxima tarefa, sessão nova, passa pela Prova, commit, avisa quando termina ou trava.
Hook	Comando que o Claude Code roda em eventos fixos, como antes de uma ferramenta.
exit 2	No `PreToolUse`, bloqueia a ferramenta e o stderr volta para o modelo.
permissions.deny	Regra do settings que o Claude Code aplica sozinho, como `Read(./.env)`.
claude -p	O Claude Code sem tela interativa: uma sessão nova por chamada.
Critério de aceite	Condição verificável que diz quando a tarefa está pronta. Vira teste.

8 Para *ir além*

- How Claude remembers your project: onde cada CLAUDE.md mora, a seção "AGENTS.md" e o import com `@`.
- Automate actions with hooks: o guia dos hooks, com o exemplo de notificação e o de bloquear comando.
- Configure permissions: regras `allow`, `ask` e `deny`, e até onde a regra de `Read` alcança.
- Run Claude Code programmatically: o `claude -p`, os modos de permissão e o `--bare`. A base do Loop.
- Building effective agents (Anthropic, 2024): retorno do ambiente a cada passo e condição de parada, o porquê da Prova dentro do Loop.

Próxima: *Aula 1.5 · Escolha sua feature-alvo*

Você já sabe qual camada mais falta no seu projeto. Falta escolher onde vai provar: uma feature pequena e verificável do seu repositório, a que o loop vai entregar no dia 7.